Автономная некоммерческая организация высшего образования
«Санкт-Петербургский институт экономики и управления»



Кафедра 09.02.07

ПРАКТИЧЕСКАЯ РАБОТА № 5

Дисциплина: МДК 03.01

Тема: Безопасный выпуск и миграция данных

Вариант № 2

Студент: Шваб Роберт Эдуардович Подпись _____

Группа: И13_24_11

Руководитель: _____ Подпись _____

Дата представления: «___» _____ 2026 г.

Оценка: _____

Санкт-Петербург 2026

СОДЕРЖАНИЕ

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ.....3
1.1 Цель и исходные данные.....3
1.2 План выпуска и возврата.....3
1.3 Контроль доступа и документы.....4
1.4 Программа и воспроизведение результата.....5
1.5 Выводы.....5
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....6
ПРИЛОЖЕНИЕ А.....7
ПРИЛОЖЕНИЕ Б.....10

1 ВЫПОЛНЕНИЕ ПРАКТИЧЕСКОЙ РАБОТЫ

1.1 Цель и исходные данные

Цель работы — спроектировать выпуск версии v2, при котором старые клиенты продолжают работать, и проверить этапы миграции на отдельной SQLite-базе.

Постановка работы приведена в перечне заданий МДК 03.01 [1].

Исходники и отчёты: репозиторий MDK0301-ServiceDesk на GitHub

1.2 План выпуска и возврата

До переключения инженер БД создаёт резервную копию, проверяет её контрольную сумму и выполняет пробное восстановление. Далее Expand добавляет priority с default 3; старые записи и вставки v1 остаются допустимыми. Контрольные точки сведены в таблицу 1 и на рисунок 1.

Таблица 1 — Контрольные точки миграции

Этап	Действие	Условие перехода
Подготовка	Бэкап и пробное восстановление	Копия открывается и проверена
Expand	Добавить priority	v1 читает и записывает
Выпуск	Развернуть v2, проверить обе версии	v1 и v2 отвечают корректно
Наблюдение	Следить за ошибками	Нет P1/P2
Contract	Удалить legacy_category	Потребителей поля нет

Источник: составлено по материалам практической работы.

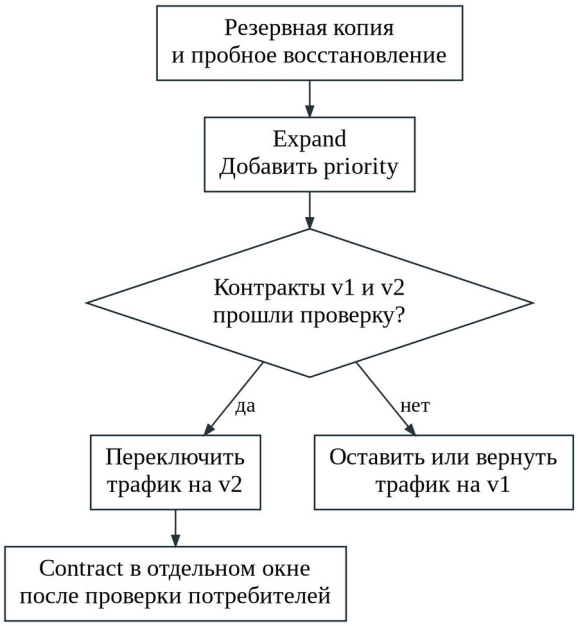


Рисунок 1 — Контрольные точки совместимой миграции

При Blue–Green старая версия продолжает обслуживать запросы, пока новая проходит проверки на совместимой схеме. Если v2 не проходит тесты, трафик возвращают на старую версию; расширенное поле сохраняют. После Contract возврат старого приложения может быть невозможен, поэтому выполняют RollForward с исправлением либо согласованное восстановление из проверенной копии.

Удаление legacy_category переносится в отдельное окно после инвентаризации потребителей поля. В ходе проверки после Expand добавлена строка старым запросом, а после Contract обе строки сохранились. SQL выполняется на тестовой базе, отдельно от API.

1.3 Контроль доступа и документы

Код и SQL меняются через pull request с просмотром миграции. Тесты контракта и миграции выполняются до переключения. Секреты хранятся вне Git, доступ к выпуску отделён от прав разработки, контрольная сумма пакета фиксируется в журнале.

Эксплуатационный комплект по структуре документации ГОСТ 34: руководство оператора — регистрация и поиск; руководство администратора — запуск, конфигурация и доступы; инструкция восстановления —

резервирование, проверка SHA-256 и порядок возврата; журнал релиза — версия, дата, SQL и ответственный; программа испытаний — сценарии и критерии.

Фактический вывод программы приведён в приложении Б. Запуск из Visual Studio: проект Verification, профиль ПР 5, Ctrl+F5.

1.4 Программа и воспроизведение результата

Проверка миграции реализована на C# в Code/Operations/Program.cs, метод Migration. SQL-сценарии выполняются Microsoft.Data.Sqlite; Python не используется. Профиль «ПР 5» выводит состояние после Expand и Contract.

Перед показом выполняется «Сборка → Пересобрать решение». Результат сборки виден в окне «Вывод → Сборка», а результат программы — в отдельной консоли Visual Studio.

1.5 Выводы

Миграция проверена на двух строках тестовой базы. Разделение этапов позволяет отменить переключение приложения без немедленного отката схемы.

Листинги приведены в приложении А.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 МДК 03.01. Перечень практических работ № 1–8. Учебное задание.

ПРИЛОЖЕНИЕ А

Исходный код программы

Code/Operations/Program.cs

```
using System.IO.Compression;
using System.Net;
using System.Security.Cryptography;
using System.Text.Json;
using Microsoft.Data.Sqlite;

try
{
    switch (args)
    {
        case ["manifest", var version, var file, var output]:
            File.WriteAllText(output, JsonSerializer.Serialize(new { Version = version, Sha256 = Hash(file) }));
            Console.WriteLine("Manifest written: " + output);
            return 0;
        case ["backup", var source, var destination]:
            if
(Path.GetFullPath(destination).StartsWith(Path.TrimEndingDirectorySeparator(Path.GetFullPath(source)) +
Path.DirectorySeparatorChar,
                OperatingSystem.IsWindows() ? StringComparison.OrdinalIgnoreCase : StringComparison.Ordinal))
                throw new ArgumentException("Archive must be outside source directory");
            ZipFile.CreateFromDirectory(source, destination);
            File.WriteAllText(destination + ".sha256", Hash(destination));
            using (var zip = ZipFile.OpenRead(destination))
                Console.WriteLine($"Backup: {zip.Entries.Count} entries; SHA-256 {Hash(destination)}");
            return 0;
        case ["sat", var address, var output]:
            return await Sat(address, output);
        case ["migration", var directory, var database]:
            return Migration(directory, database);
        default:
            Console.Error.WriteLine("Operations: manifest VERSION FILE JSON | backup SOURCE ZIP | sat URL
MARKDOWN | migration SQL_DIR DB");
            return 1;
    }
}
catch (Exception error)
{
    Console.Error.WriteLine(error.Message);
    return 1;
}

static string Hash(string path)
{
    using var stream = File.OpenRead(path);
    return Convert.ToHexString(SHA256.HashData(stream));
}
static async Task<int> Sat(string address, string output)
{
    using var http = new HttpClient { BaseAddress = new Uri(address), Timeout = TimeSpan.FromSeconds(5) };
    var checks = new List<(string Scenario, bool Passed, string Detail)>();
    foreach (var (route, expected) in new[]
    {
```

```
    ("/health", HttpStatusCode.OK),
    ("/api/v1/tickets", HttpStatusCode.OK),
    ("/api/v2/tickets", HttpStatusCode.OK),
    ("/api/v1/tickets/2147483647", HttpStatusCode.NotFound)
  })
  {
    try
    {
      using var response = await http.GetAsync(route);
      var correctBody = true;
      if (expected != HttpStatusCode.NotFound)
      {
        using var json = JsonDocument.Parse(await response.Content.ReadAsStringAsync());
        correctBody = route == "/health" ? json.RootElement.GetProperty("status").GetString() == "ok"
          : json.RootElement.ValueKind == JsonValueKind.Array;
      }
      checks.Add((route, response.StatusCode == expected && correctBody, $"HTTP
{(int)response.StatusCode}"));
    }
    catch (Exception e) when (e is HttpRequestException or TaskCanceledException or JsonException or
KeyNotFoundException or InvalidOperationException)
    { checks.Add((route, false, e.GetType().Name)); }
  }
  foreach (var x in checks) Console.WriteLine($"{(x.Passed ? "PASS" : "FAIL")}: GET {x.Scenario} —
{x.Detail}");
  var passed = checks.Count(x => x.Passed);
  var lines = new List<string> { "# Протокол SAT — стенд", "", $"Дата: {DateTimeOffset.UtcNow:yyyy-MM-
dd HH:mm:ss} UTC", "",
    "| Сценарий | Результат | Ответ |", "| --- | --- | --- |" };
  foreach (var x in checks) lines.Add($"| GET {x.Scenario} | {(x.Passed ? "PASS" : "FAIL")} | {x.Detail} |");
  lines.Add($"\nПройдено: {passed}/{checks.Count}. Согласование заказчиком проводится отдельно.");
  File.WriteAllLines(output, lines);
  Console.WriteLine($"SAT: {passed}/{checks.Count} PASS");
  return passed == checks.Count ? 0 : 2;
}
static int Migration(string directory, string database)
{
  if (File.Exists(database)) throw new ArgumentException("Use a new database file for the migration
demo");
  using var connection = new SqliteConnection(new SqliteConnectionStringBuilder { DataSource =
database }.ToString());
  connection.Open();
  void Exec(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql;
cmd.ExecuteNonQuery(); }
  long Read(string sql) { using var cmd = connection.CreateCommand(); cmd.CommandText = sql; return
Convert.ToInt64(cmd.ExecuteScalar()); }
  Exec(File.ReadAllText(Path.Combine(directory, "schema_v1.sql")));
  Exec("INSERT INTO tickets (title,status,legacy_category) VALUES ('Existing ticket','Open','Hardware')");
  Exec(File.ReadAllText(Path.Combine(directory, "migrate_expand.sql")));
  Exec("INSERT INTO tickets (title,status) VALUES ('Old client ticket','Open')");
  if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2) throw new
InvalidOperationException("Expand failed");
  Console.WriteLine("PASS: original row preserved and v1 insert works after Expand");
  Console.WriteLine("After Expand: 2 rows, priority=3; INSERT v1 works.");
  Exec(File.ReadAllText(Path.Combine(directory, "migrate_contract.sql")));
  if (Read("SELECT COUNT(*) FROM tickets WHERE priority=3") != 2 ||
    Read("SELECT COUNT(*) FROM pragma_table_info('tickets') WHERE name='legacy_category'") != 0)
    throw new InvalidOperationException("Contract failed");
  Console.WriteLine("PASS: Contract removes the legacy column and keeps both rows");
```

```
    Console.WriteLine("After Contract: 2 rows; legacy_category absent.");
    return 0;
}
```

Code/Operations/Operations.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup><OutputType>Exe</OutputType><TargetFramework>net8.0</
TargetFramework><ImplicitUsings>enable</ImplicitUsings><Nullable>enable</Nullable></PropertyGroup>
  <ItemGroup><PackageReference Include="Microsoft.Data.Sqlite" Version="8.0.11"/></ItemGroup>
</Project>
```

Code/sample-data/schema_v1.sql

```
CREATE TABLE tickets (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    status TEXT NOT NULL,
    legacy_category TEXT
);
```

Code/sample-data/migrate_expand.sql

```
BEGIN;
ALTER TABLE tickets ADD COLUMN priority INTEGER NOT NULL DEFAULT 3 CHECK(priority BETWEEN 1
AND 4);
COMMIT;
```

Code/sample-data/migrate_contract.sql

```
-- Apply only after clients no longer depend on legacy_category.
BEGIN;
ALTER TABLE tickets DROP COLUMN legacy_category;
COMMIT;
```

ПРИЛОЖЕНИЕ Б

Фактический вывод контрольного прогона

```
PASS: original row preserved and v1 insert works after Expand
After Expand: 2 rows, priority=3; INSERT v1 works.
PASS: Contract removes the legacy column and keeps both rows
After Contract: 2 rows; legacy_category absent.
```

Снимки выполнения программы в Visual Studio

Я запустил программы в Visual Studio под Windows. На снимках приведены исходный код и результаты контрольного запуска.

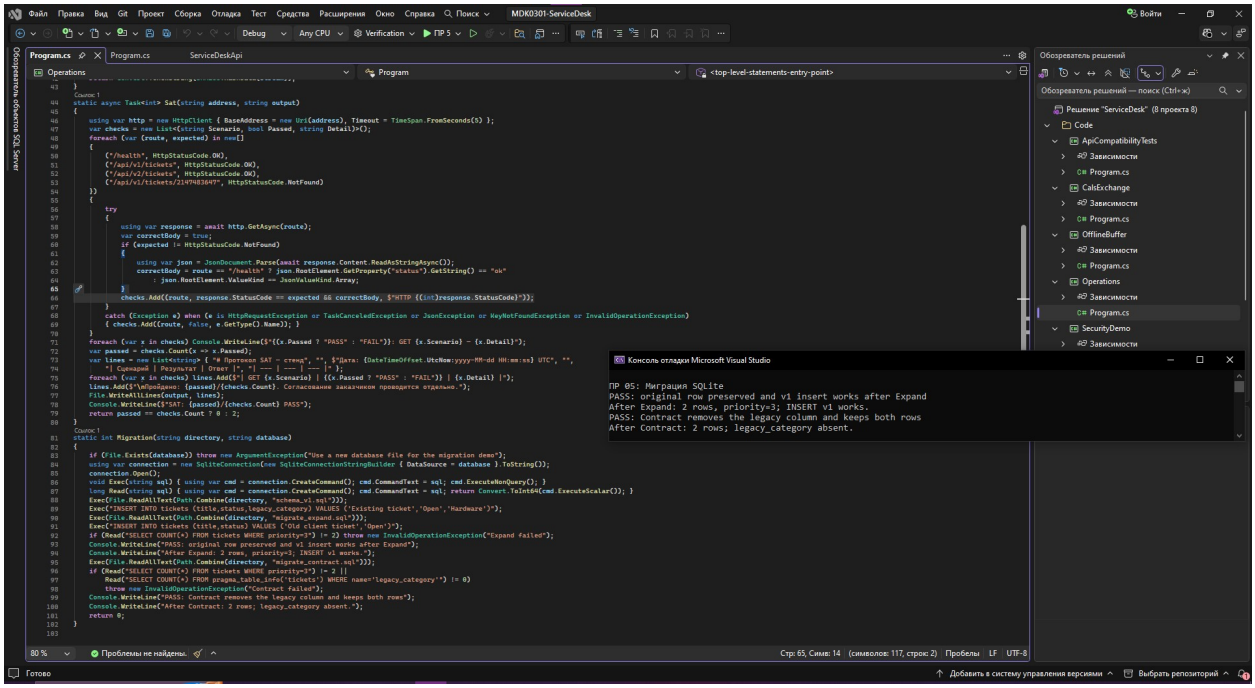


Рисунок Б.1 — Миграция SQLite по схеме Expand–Contract. После обоих этапов сохранены две записи; устаревший столбец удалён.